

Radar Inference *Accelerator*

A custom silicon tapeout combining matched-filter DSP and neural inference — submitted to the Spring 2027 Sky130 MPW shuttle.

256

MACs / cycle

16×16

Systolic array

Sky130

Open-source PDK

~12 mo

RTL → tapeout

What we're building

A complete radar signal processing pipeline on a single chip.

Raw radar returns enter the matched filter front end, get compressed into a feature vector, and feed directly into a 16×16 systolic array that runs INT8 neural inference. The output is a target classification — range, type, or both — produced in real time.

An on-chip RISC-V host (PicoRV32) drives the whole thing over Wishbone. **No external FPGA. No external microcontroller. Just silicon.**

PIPELINE

Radar samples

complex IQ input

Matched Filter

FFT • multiply • IFFT

Systolic Array

16×16 INT8 MACs

Activation

ReLU + requantize

Classification

argmax → target class

Where this chip would be used

Real-time radar classification is latency-critical. CPUs can't keep up; GPUs burn too much power. Custom silicon is the right answer.



Autonomous vehicles

Detect and classify pedestrians, cyclists, and vehicles in adverse weather where cameras fail.



Drone detection

Distinguish drones from birds at long range — a hard classification problem solved at the edge.



Satellite SAR

On-orbit synthetic aperture radar processing without downlinking raw data to ground stations.



Research platform

Underexplored territory: open-source matched-filter + inference on one die. Publishable.

None of these can rely on a server in the loop. The compute has to live next to the antenna.

What's new about it

First

open-source tapeout combining a matched-filter DSP front end with a systolic-array NN backend on a single die.

Most accelerators do one or the other. We're doing both — and releasing the GDS.

01 Both stages, one die

DSP and inference share clock, fabric, and memory — no off-chip hop between pulse compression and classification.

02 Fully parameterized RTL

FFT size, array dimensions, and quantization width are compile-time knobs. Reusable for follow-on tapeouts.

03 Open toolchain end-to-end

Yosys, OpenLane, OpenROAD, Magic, KLayout. No commercial license walls. Reproducible by anyone.

04 Path to NYU research collaboration

RF and wireless groups have direct interest. Real route toward grant-funded extensions.

Five blocks on one die

Sitting inside the Efabless Caravel harness — PicoRV32 host, Wishbone bus, and clock management come for free.



Matched Filter Front End

Pipelined FFT → complex multiply → IFFT. Pulse compression in fixed-point. Target 64-point.



16×16 Systolic Array

256 PEs, each one INT8 multiplier + INT32 accumulator. 256 MACs every cycle.



Activation Unit

Combinational. Q16.16 requantize, INT8 clip, ReLU. Small area, easy to verify.



Wishbone Slave Interface

Memory-mapped registers. Host writes weights and inputs, polls done, reads result.



Control FSM

Orchestrates: load weights → load samples → matched filter → systolic → activate → write back.



Caravel Harness (provided)

PicoRV32 RISC-V host, Wishbone bus, GPIO, clock management. No external MCU needed.

The math, in two stages

Every block on the previous slide exists to compute one of these two equations as fast as possible.

STAGE 1 / MATCHED FILTER FRONT END

$$X_{\text{compressed}} = \text{IFFT}(\text{FFT}(\text{signal}) \cdot \text{conj}(\text{FFT}(\text{chirp})))$$

1 *FFT*

Transform incoming complex radar samples to the frequency domain.

2 *Multiply*

Pointwise against the conjugate of the stored reference chirp spectrum.

3 *IFFT*

Transform back. Output is a range-compressed profile — the peak locates the target.

The output becomes the feature vector for Stage 2.

STAGE 2 / NEURAL NETWORK INFERENCE

$$y = \text{ReLU}(W \cdot X_{\text{compressed}} + b)$$

1 *Matmul*

W is the INT8 weight matrix. The 16×16 systolic array computes W·x in parallel — 256 MACs/cycle.

2 *Activate*

ReLU plus Q16.16 requantization brings INT32 accumulator output back to INT8.

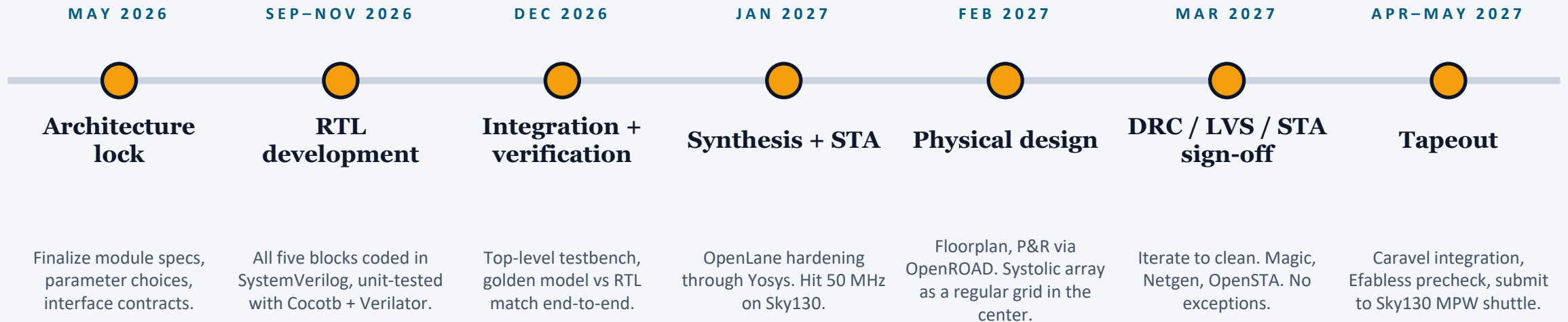
3 *Argmax*

Largest output element is the predicted target class. That's the chip's answer.

The output is the target classification.

The next twelve months

From here to a Spring 2027 Sky130 shuttle. Each phase has a clean handoff.



BACKUP PLAN If Sky130 shuttle timing slips, fall back to GlobalFoundries 180nm via IHP. Same flow, longer pipeline.

How we split the work

Four subgroups, each owning a clean slice of the chip. Members can rotate after the RTL phase.



DSP Group

OWNS

Matched filter front end

- FFT / IFFT butterfly hardware
- Complex multiply stage
- Reference chirp storage
- Fixed-point precision tuning

SKILLS

DSP fundamentals, fixed-point math, SystemVerilog



Accelerator Group

OWNS

Systolic array + activation

- 256-PE grid design
- INT8 multiplier + INT32 accumulator
- Weight-stationary dataflow
- ReLU + requantization unit

SKILLS

Computer arch, ML quantization, RTL



Integration Group

OWNS

Glue, FSM, host interface

- Wishbone slave interface
- Top-level control FSM
- Caravel harness wiring
- Memory map + register file

SKILLS

Bus protocols, FSM design, system integration



Verification + Backend

OWNS

Testing, synthesis, P&R

- Cocotb testbenches per block
- Python golden model (NumPy)
- OpenLane synthesis flow
- DRC / LVS / STA sign-off

SKILLS

Python, Verilator, OpenLane, EDA tools

How we get from RTL to GDS



Each step gates the next. We don't move forward until the current stage is signed off.

Open-source from RTL to GDS

No commercial license walls. Every tool below is free, scriptable, and runs on a laptop.

DESIGN

RTL & specification

SystemVerilog

Hardware description language. Every block is written in SV.

Caravel harness

Provided by Efabless. Wraps our user area with PicoRV32, Wishbone, GPIO.

Git + GitHub

Version control. PRs reviewed across subgroups before merging.

VERIFICATION

Functional correctness

Cocotb

Python-based testbench framework. Stimulus and checking in NumPy.

Verilator

Fast open-source simulator. Drives Cocotb at full speed.

Python + NumPy

Golden model. RTL output must match bit-for-bit within quantization tolerance.

BACKEND

Synthesis through tapeout

Yosys

Logic synthesis. RTL → gate-level netlist on Sky130 standard cells.

OpenLane

Wraps the full flow: synth, floorplan, P&R, sign-off in one command.

OpenROAD

Place and route. Built into OpenLane. Handles the systolic grid placement.

Magic + KLayout

Layout viewers and DRC. Final visual sanity check before tapeout.

Netgen + OpenSTA

LVS (schematic vs layout) and static timing analysis at sign-off.

Known risks and how we handle them

Sky130 shuttle slip

MPW shuttle dates shift. Fallback: IHP GlobalFoundries 180nm via the same open flow. RTL is PDK-agnostic by design.

Timing closure at 50 MHz

INT8 multiplier is likely the critical path. Mitigation: pipeline the multiplier, or target 25 MHz with minimal throughput impact.

Team continuity

Members graduate or drop. Mitigation: fully documented RTL; subgroup leads train backups before Jan 2027 integration phase.

Area blowout

Design exceeds Caravel user area. Mitigation: parameterize down to 8x8 array and 32-point FFT; full architecture retained, smaller first silicon.

Verification gaps

RTL bugs reaching tapeout. Mitigation: all blocks gate on passing against the NumPy golden model before moving to integration.

Why nothing else does what this chip does

APPROACH	DSP FRONT END	NN INFERENCE	OPEN SOURCE	SINGLE DIE
GPU (e.g. Jetson Orin)	Software only	Yes	No	No
FPGA SoC (Xilinx RFSoc)	Yes	Partial	No	No
Research ASICs (closed-source)	Some	Some	No	No
This project (Sky130 tapeout)	Yes	Yes	Yes	Yes

No existing open-source tapeout combines a matched-filter DSP front end with a systolic-array NN backend on one die. That is the gap this project fills.

Let's tape out *real silicon.*

If we lock the architecture in May, we have twelve clean months to first silicon. The flow is open-source. The shuttle is funded. The novelty is real.

TONIGHT

Pick subgroup leads

THIS WEEK

**Lock FFT size + array
dim**

END OF MAY

**Architecture frozen, RTL
begins**